TP Algorithmique : système de livraison

Adam SIDHOUM

Langage

J'ai choisi le C, le langage de notre cours. Comme il ne fournit ni liste dynamique ni dictionnaire, j'ai dû écrire moi-même chaque structure, ce qui rend leurs coûts faciles à justifier.

Les colis sont stockés une seule fois dans un tableau. Les autres structures (table de hachage, tri, chargement) ne manipulent que des indices vers ce tableau. Convention de priorité : 1 = faible, 5 = la plus importante.

Choix 1 : stocker les colis

Problème	Stocker un nombre de colis inconnu à l'avance, jusqu'à 100 000.
Solution	Tableau dynamique : quand il est plein, on double sa capacité avec `realloc`.
Temps	Ajout en O(1) amorti, accès par indice en O(1).
Espace	O(n).
Alternative	Liste chaînée : ajout en O(1), mais accès en O(n).
Justification	En doublant, le total des recopies reste inférieur à 2n. Augmenter de 1 à chaque fois coûterait $O(n^2)$ au total.

Choix 2 : rechercher un colis par identifiant

Problème	Retrouver rapidement un colis par son ID, et refuser les ID en double.
Solution	Table de hachage par chaînage (ID vers indice), fonction `id mod m` avec m premier, agrandie quand elle est remplie à plus de 75 %.
Temps	O(1) en moyenne, O(n) dans le pire cas.
Espace	O(n).
Alternative	Parcours linéaire en O(n), ou tableau trié avec dichotomie en O(log n), mais l'ajout coûte alors O(n).
Justification	Les recherches sont fréquentes. Le pire cas est très rare : sur 100 000 colis, la plus longue chaîne contient 7 éléments.

Choix 3 : trier par poids, valeur ou priorité

Problème	Afficher les colis triés selon trois critères.
Solution	Tri fusion sur un tableau d'indices, avec une seule fonction qui reçoit un comparateur en paramètre.
Temps	O(n log n) dans tous les cas.
Espace	O(n).
Alternative	Tri par insertion en $O(n^2)$, ou tri rapide, en $O(n^2)$ dans le pire cas.
Justification	La complexité est garantie et le tri est stable. Trier des indices évite de casser la table de hachage.

Choix 4 : remplir le camion

Le glouton (prendre les colis par meilleur rapport valeur/poids) n'est pas toujours optimal. Par exemple, avec une capacité de 10 kg et les colis (6 kg, 30), (5 kg, 20) et (5 kg, 20), il prend le premier et obtient 30, alors que les deux autres donnent 40.

Problème	Choisir les colis qui maximisent la valeur sans dépasser la capacité W.
Solution	Sac à dos 0/1 par programmation dynamique : `dp[w] = max(dp[w], dp[w - poids] + valeur)`. Un tableau de bits mémorise les choix pour retrouver les colis chargés.
Optimal ?	Oui, toujours. Je l'ai vérifié contre la force brute sur 500 cas aléatoires.
Temps	O(n × W).
Espace	O(W) + n × W bits.
Alternative	Force brute en $O(2^n)$, ou glouton, rapide mais approché (optimal dans seulement 57 % de mes tests).
Justification	C'est la seule méthode exacte et rapide pour des capacités réalistes. Sa limite : le coût dépend de la valeur de W. Un très grand W devient trop coûteux en mémoire, et le programme refuse alors le calcul.

Prise en compte de la priorité

Je garde le même algorithme et je remplace seulement la valeur par un score :

$$score = valeur \times (100 + \alpha \times (priorité - 1))$$

Avec α = 0, la priorité est ignorée. Avec α = 25, un colis de priorité 5 compte deux fois sa valeur. Le poids reste la contrainte de capacité, la valeur reste la base du score, et α permet de régler l'importance de la priorité. Sur l'exemple de l'énoncé, avec un camion de 15 kg :

- α = 0 : B + C + D (valeur 57) ;
- α = 50 : A + C (valeur 38). B, de priorité 1, est écarté au profit de A et C, de priorités 5 et 4.

Évolution quand n augmente

Temps mesurés pour 10 000 recherches et pour un tri complet :

n	Recherche linéaire	Recherche hachage	Tri insertion	Tri fusion
1 000	3 ms	0,08 ms	0,4 ms	0,09 ms
10 000	32 ms	0,25 ms	45 ms	1,2 ms
100 000	724 ms	0,5 ms	8 900 ms	13 ms

- **Recherche.** Le temps de la recherche linéaire est multiplié par environ 10 quand n est multiplié par 10, comme prévu pour O(n). Avec la table de hachage, il reste presque constant. Le choix reste adapté.
- **Tri.** Le tri par insertion est multiplié par environ 100 quand n est multiplié par 10, comme prévu pour $O(n^2)$, et devient inutilisable. Le tri fusion suit O(n log n). Amélioration possible : un tri par comptage pour la priorité, qui n'a que 5 valeurs.
- **Chargement.** Pour W = 50 kg, il faut 5,5 ms avec 100 000 colis. En revanche, le temps augmente proportionnellement à W, c'est la vraie limite. Si W devenait très grand, il faudrait exprimer les poids dans une unité plus grosse ou utiliser une méthode approchée.